

Description:

Sum the file is CMPE 310 Spring 2025 second assembly language project. Students had to write an x86 NASM assembly program that reads a set of integers from a file into an array. The data file has to be read from the command line. After doing so, add up all the numbers and display the sum on the screen.

(PS, I was sick for two of the labs before the due date of this assignment, so I was unable to attend lab)

Here is how I decided to tackle my problem, I broke it up into parts. The first part was to get the user data, AKA the file name, then open up the file. After opening up the file, I read it and stored it in the array. After I had to convert the strings into integers, then I added the integer to an accumulator to keep track of the sum. After all calculations were made, I printed out the sum.

Here is the code:

```
Unset
section .data
    prompt: dd "Enter the name of the file: "
    promptLen equ $ - prompt

section .bss
    buffer: resb 1024
    num_buffer resb 12
    pathname resb 128

global main
section .text
main:
    ;Prompting the user
    mov eax, 4
    mov ebx, 1
    mov ecx, prompt
    mov edx, promptLen
    int 80h

    ;Getting the file name
    mov eax, 3
    mov ebx, 0
    mov ecx, pathname
    mov edx, 128
    int 80h
    mov esi, eax
```

```

dec esi
mov byte[pathname + esi], 0

;Opening the file
mov eax, 5
mov ebx, pathname
mov ecx, 0
int 80h
mov ebx, eax

;Reading the file into the buffer
mov eax, 3
mov ecx, buffer
mov edx, 1024
int 80h
mov edi, eax

xor eax, eax ;Holds the sum
xor ebx, ebx ;Current Value
xor ecx, ecx ;Index
xor edx, edx
xor esi, esi ;Line counter

```

parse:

```

cmp ecx, edi
jge done_parse

mov dl, [buffer + ecx]
cmp dl, 10
je new_line

cmp dl, 13
je next

cmp dl, '0'
jb next
cmp dl, '9'
ja next

sub dl, '0'
imul ebx, ebx, 10
add ebx, edx
jmp next

```

```

new_line:
    inc esi
    cmp esi, 1
    je reset
    add eax, ebx

reset:
    xor ebx, ebx

next:
    inc ecx
    jmp parse

done_parse:
    cmp ebx, 0
    je exit
    cmp esi, 0
    jle exit
    add eax, ebx

exit:
    call print_answer
    mov eax, 1
    int 80h

print_answer:
    mov ecx, num_buffer
    add ecx, 11
    mov byte [ecx], 10
    dec ecx

    cmp eax, 0
    jne convert
    mov byte [ecx], '0'
    jmp print

convert:
next_digit:
    xor edx, edx
    mov ebx, 10
    div ebx
    add dl, '0'
    mov [ecx], dl

```

```
dec ecx
test eax, eax
jnz next_digit
```

```
print:
    mov eax, 4
    mov ebx, 1
    mov edx, num_buffer + 12
    sub edx, ecx
    mov ecx, ecx
    int 80h
```

Provided File for Testing:

Name: randomInt100.txt

Unset

```
100
85
13
90
32
4
84
9
92
26
10
34
46
99
36
61
36
86
92
17
76
83
58
3
43
4
```

26
55
28
36
46
82
3
100
22
6
62
23
40
26
32
3
48
100
93
72
81
89
42
21
91
26
47
81
54
91
76
23
42
24
38
57
34
2
12
15
67
45
79
85

```
70
75
20
3
67
19
24
11
96
91
31
82
70
47
63
45
3
83
12
17
14
16
18
67
14
58
7
13
32
17
50
```

My File for Testing:

Name: mytest.txt

Unset

```
25
1
2
3
4
```

```
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
```

Output:

Filename: randomInt100.txt

```
[aamuah1@linux5 Sum_File]$ nasm -f elf64 sum1.asm
[aamuah1@linux5 Sum_File]$ gcc -o sum1 sum1.o
[aamuah1@linux5 Sum_File]$ sum1
Enter the name of the file: randomInt100.txt
4579
```

Filename: mytest.txt

```
[aamuah1@linux5 Sum_File]$ nasm -f elf64 sum1.asm
[aamuah1@linux5 Sum_File]$ gcc -o sum1 sum1.o
[aamuah1@linux5 Sum_File]$ sum1
Enter the name of the file: mytest.txt
325
```